Lecture 02

Software Re-Engineering

Engr. Maqsood Khatoon

Department of Software Engineering

National University of Computer & Emerging Sciences

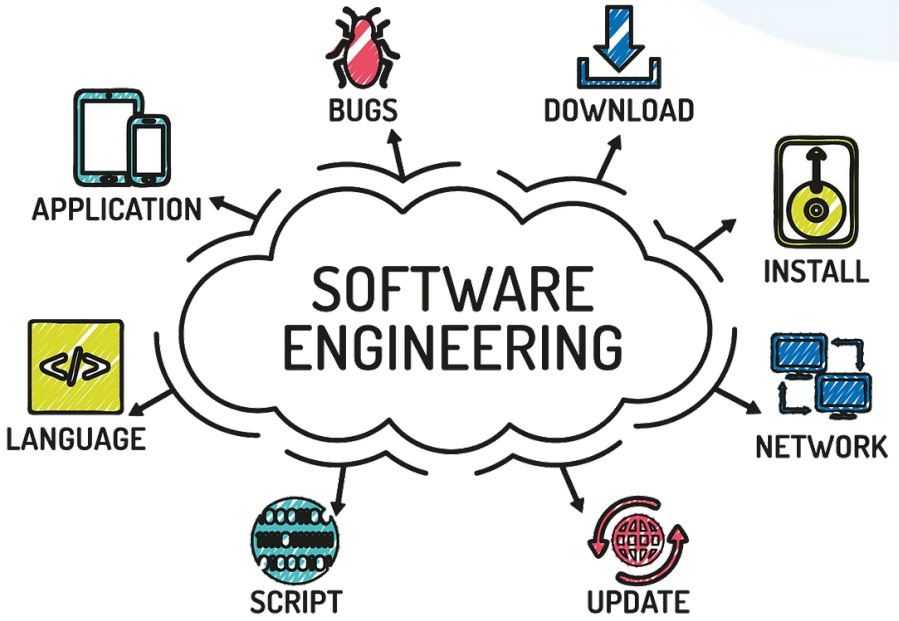
maqsood.khatoon@nu.edu.pk

Lecture 02

Introduction to Software Engineering

WHAT IS SOFTWARE ENGINEERING?

- The term software engineering is composed of two words, software and engineering.
- **Software** is more than just a program code. A program is an **executable code**, which serves some computational purpose. **Software** is considered to be a collection of **executable programming code**, **associated libraries** and **documentations**. Software, when made for a specific requirement is called software product.
- **Engineering** on the other hand, is all about developing products, using well-defined, scientific principles and methods.

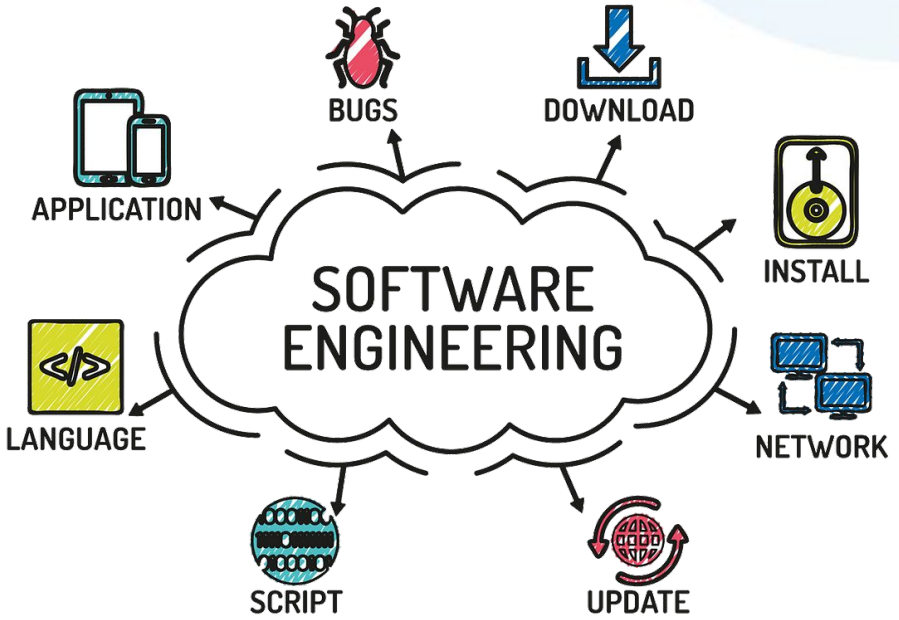


WHAT IS SOFTWARE ENGINEERING?

1. The IEEE Definition (Standard)

The **Institute of Electrical and Electronics Engineers (IEEE)** provides the most frequently cited technical definition. It emphasizes that software development must be measurable and structured.

> **Definition:** "The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software."
>
> **Reference:** *IEEE Standard Glossary of Software Engineering Terminology*, IEEE Std 610.12-1990.

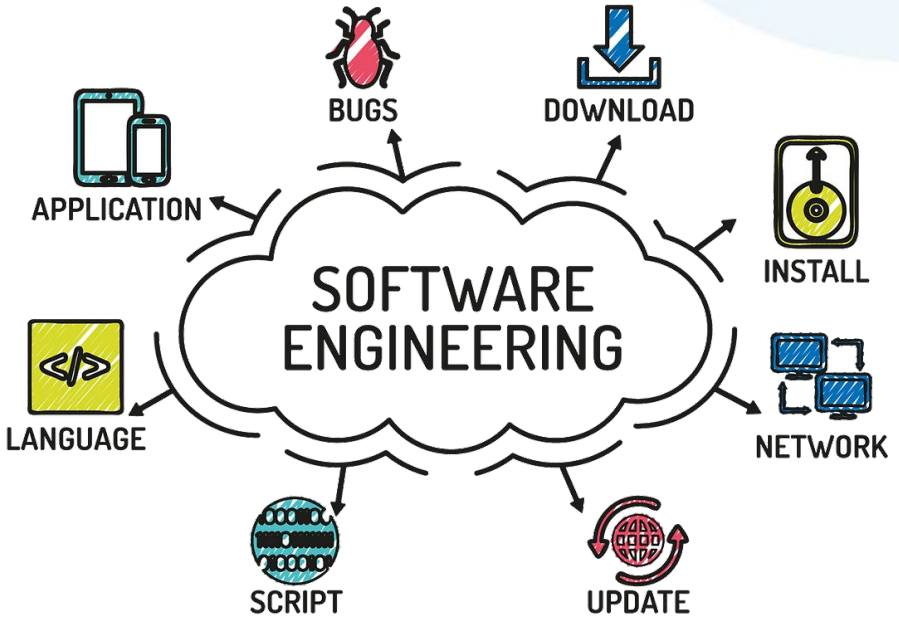


WHAT IS SOFTWARE ENGINEERING?

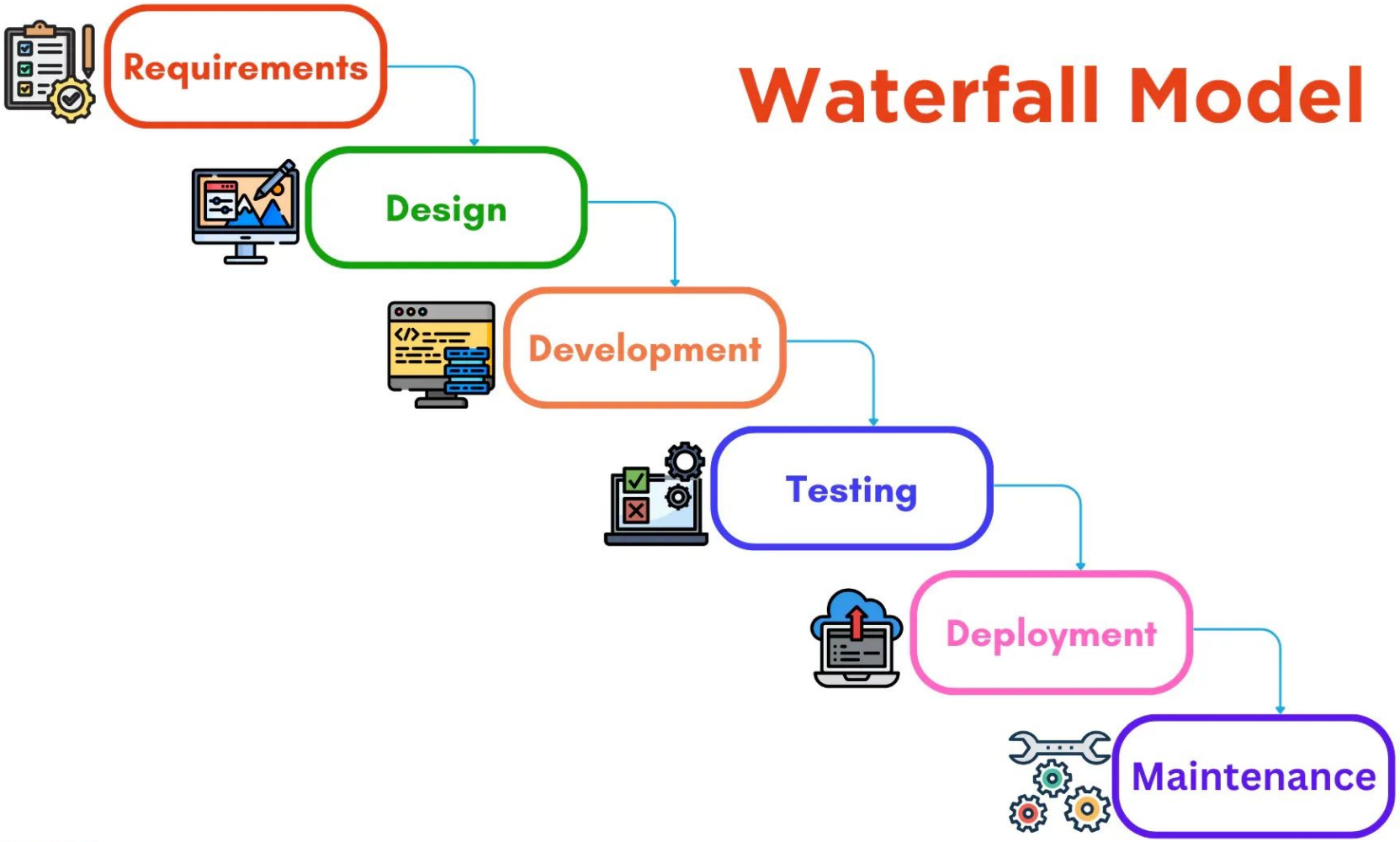
- 2. Software Engineering as a "Layered Technology"

- In the industry-standard textbook by **Roger S. Pressman**, software engineering is described not just by a sentence, but as a framework consisting of four layers.

Layer	Purpose
Quality Focus	The bedrock of any engineering project; the commitment to continuous improvement.
Process	The "glue" that holds the layers together; it defines the workflow (e.g., Agile, Waterfall).
Methods	The "how-to" for building software (e.g., requirements analysis, design, testing).
Tools	Automated or semi-automated support for the layers above (e.g., IDEs, Git, JIRA).



Software Development Life Cycle-SDLC



Deliverables of each phase of SDLC

Requirements

Project Charter, Feasibility Study, High-Level Project Plan, Resource Allocation.
Detailed Requirements Document

Design

Software Design Document (SDD), Architecture Diagrams, UI/UX Mockups, Database Schemas.

Development

Source Code, Functional Prototypes, Unit Test Reports, Technical Documentation.

Testing

Test Plans, Test Cases, Bug Reports, Quality Assurance Reports, Regression Test Results.

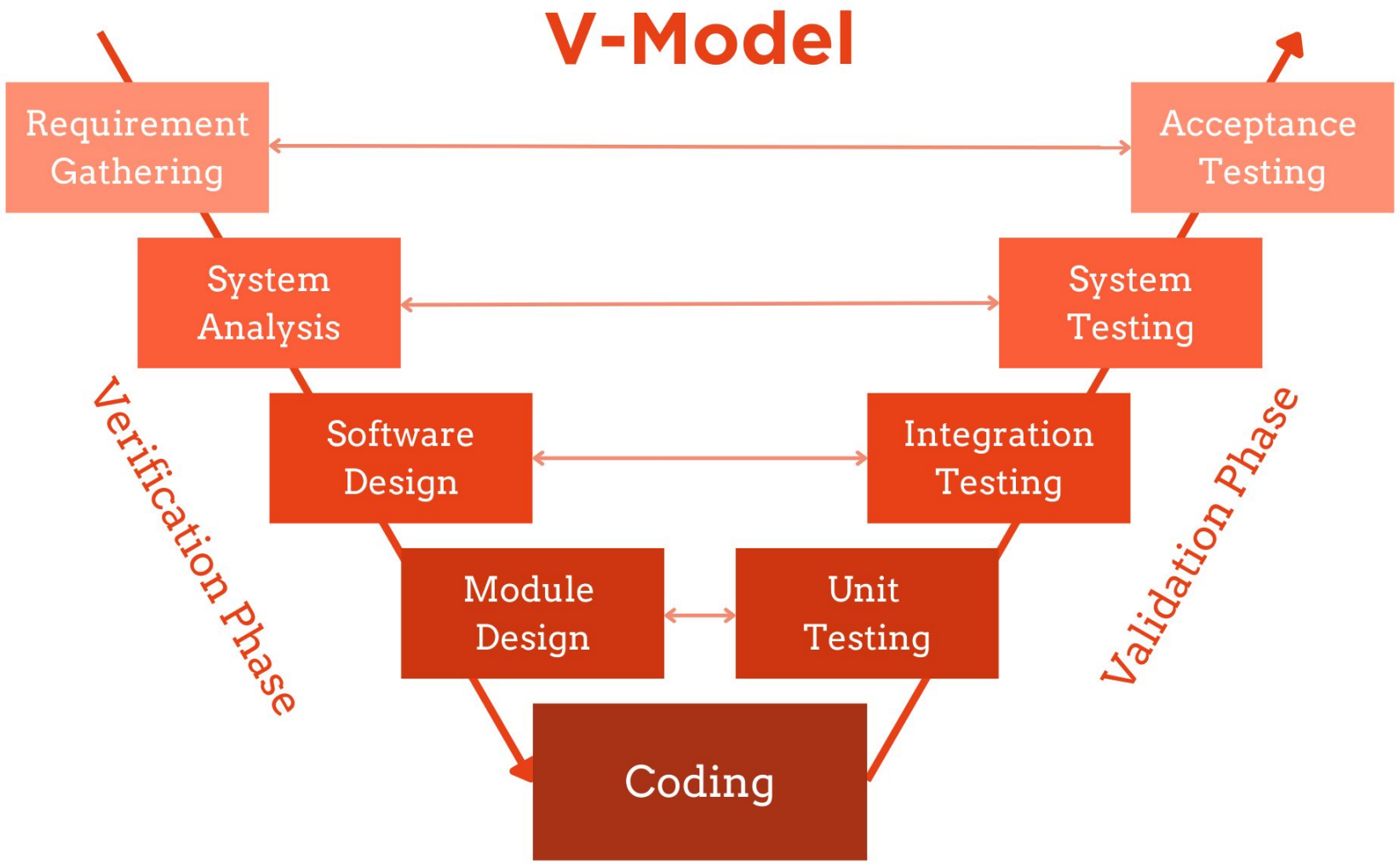
Deployment

Release Notes, User Manuals, Production Environment Setup, Final Packaged Software.

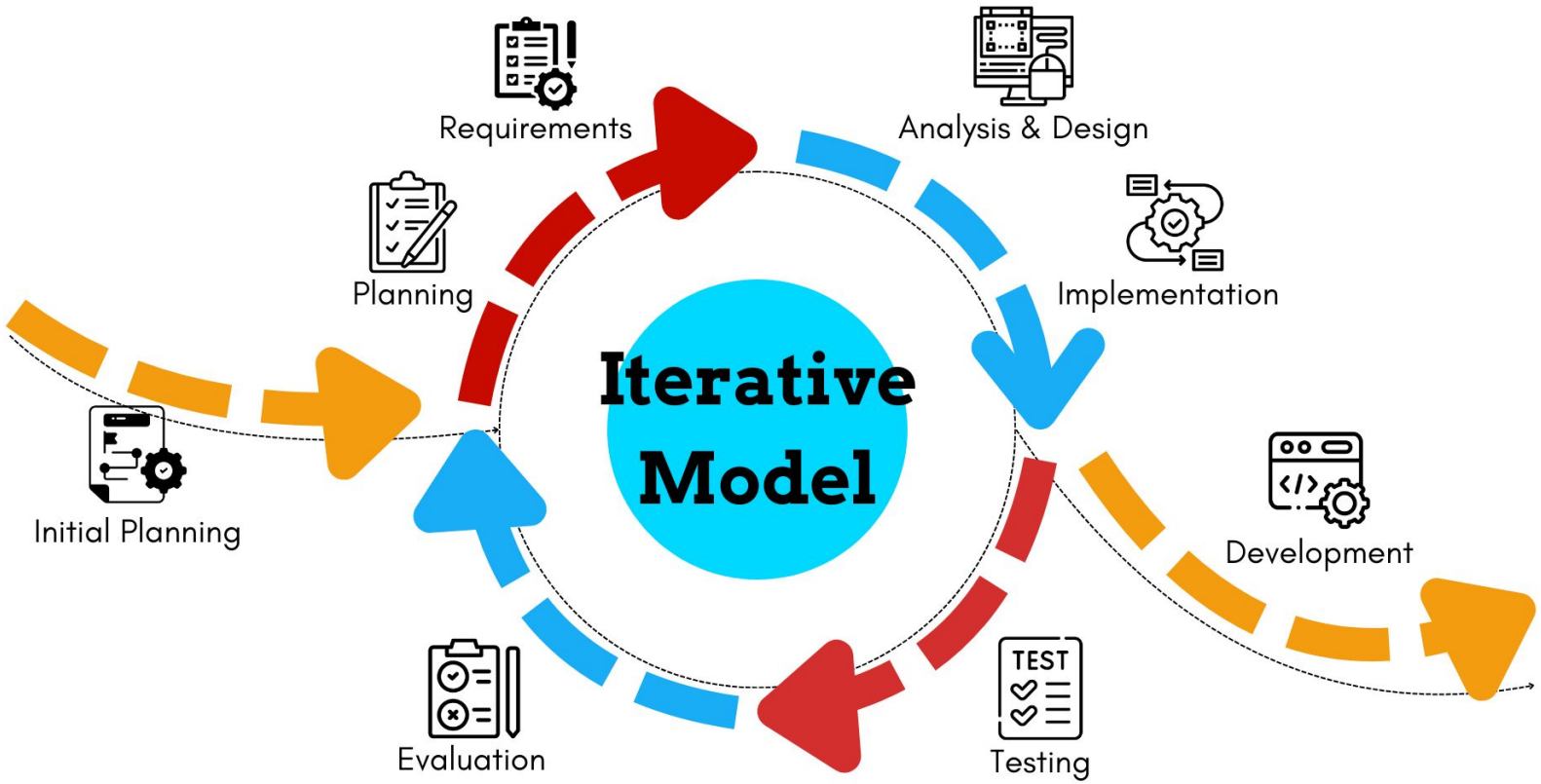
Maintenance

Patches, Updates, Performance Reports, User Support Documentation, Enhanced Code.

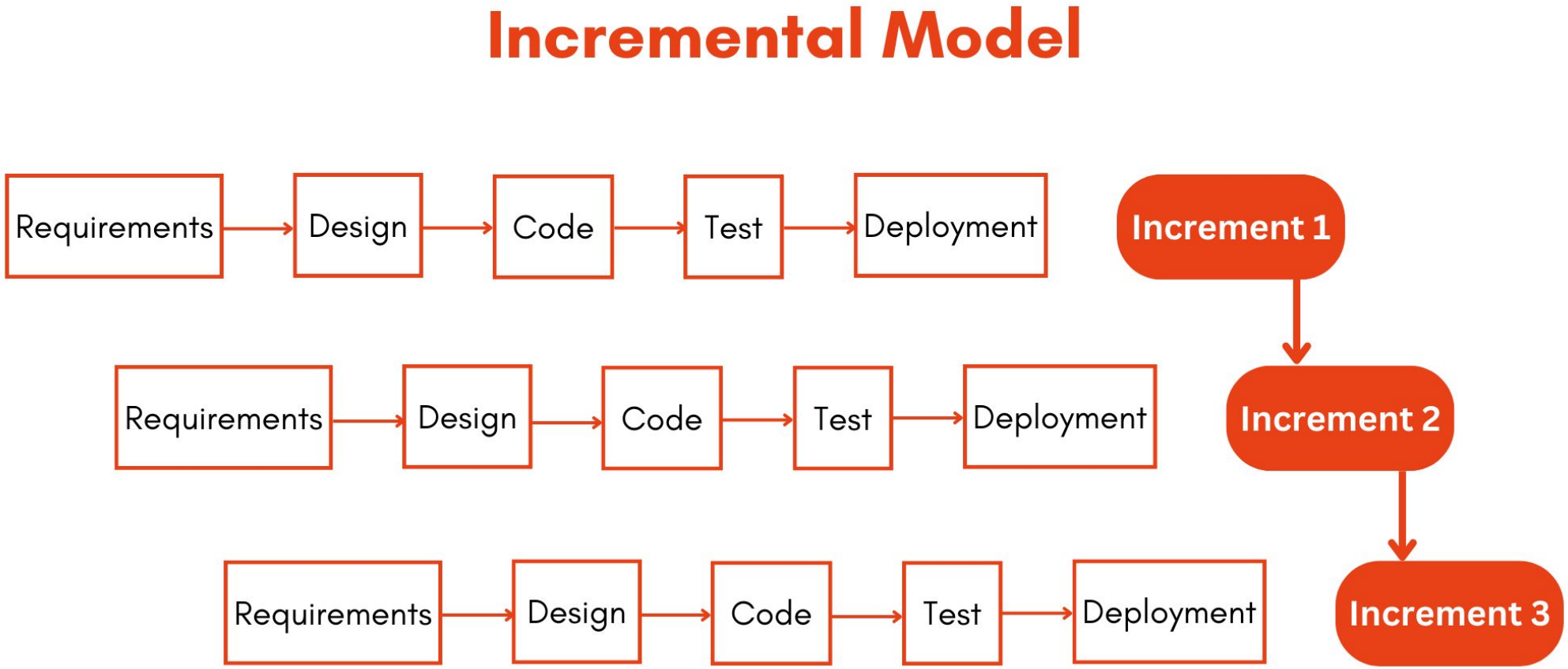
Software Development Life Cycle-SDLC



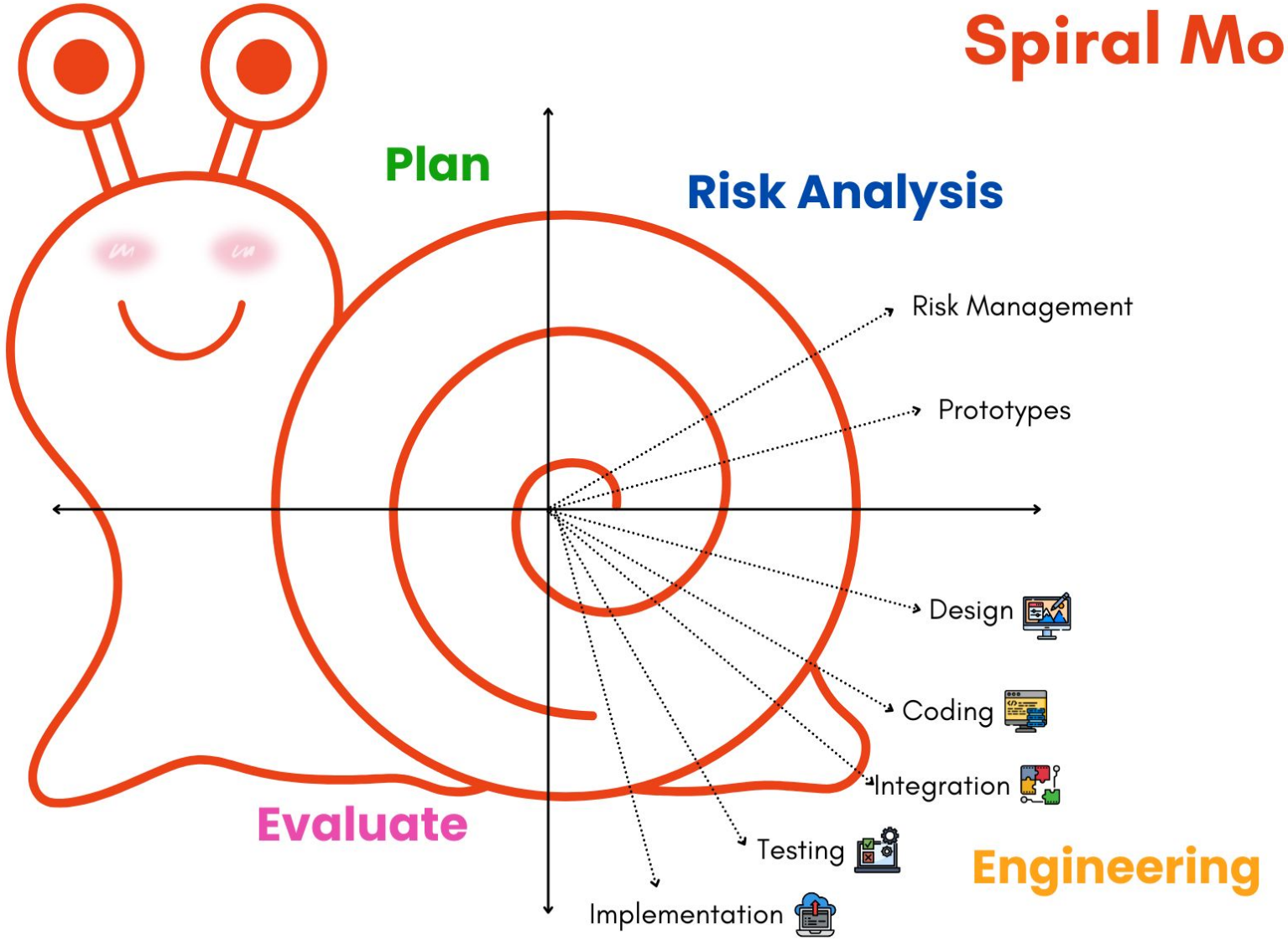
Software Development Life Cycle-SDLC



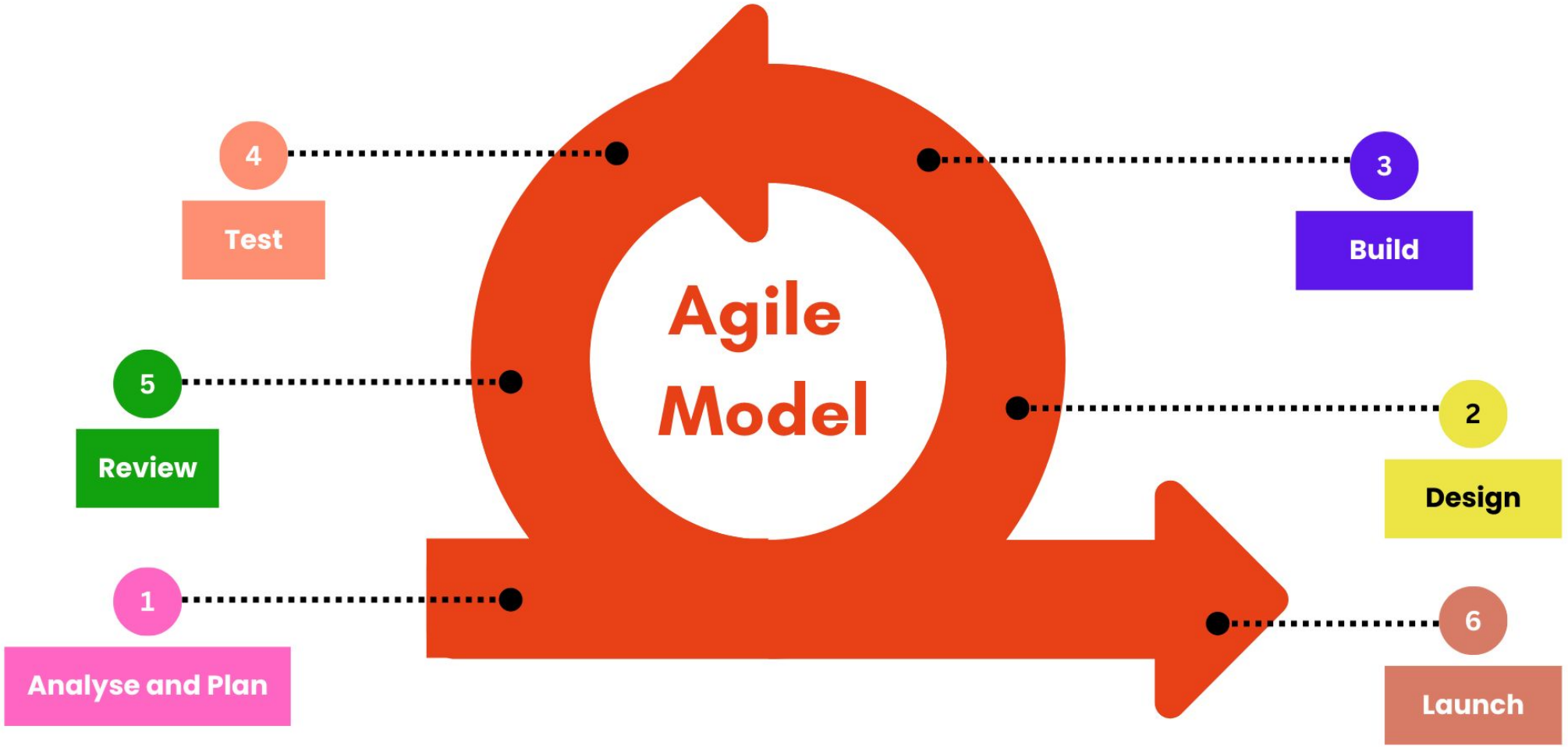
Software Development Life Cycle-SDLC



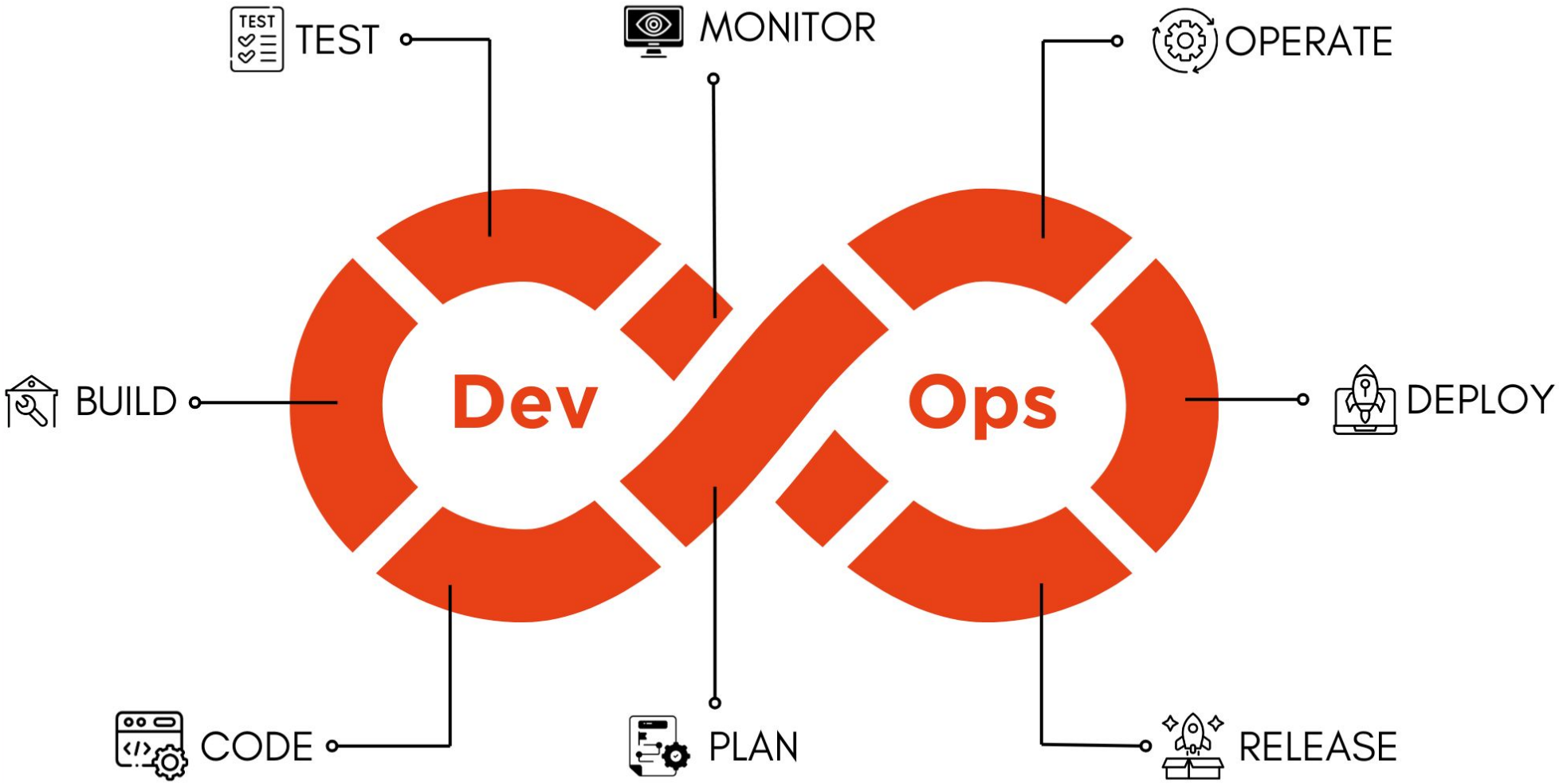
Software Development Life Cycle-SDLC



Software Development Life Cycle-SDLC



Software Development Life Cycle-SDLC



Factor	Waterfall Model	V-Model	Iterative Model	Incremental Model	Spiral Model	Agile Model	Lean Model	DevOps Model
Approach	Linear and Sequential	Linear with Verification and Validation	Repeated Cycles	Successive Increments	Risk-Driven Cycles	Iterative and Incremental	Value and Waste Minimization	Integration of Development and Operations
Flexibility	Low	Low	High	Medium	High	Very High	High	High
Customer Involvement	Low	Low	Medium	High	High	Very High	High	High
Risk Management	Low	Medium	Medium	Medium	Very High	Medium	High	High
Documentation	Extensive	Extensive	Varies	Varies	Varies	Light to Medium	Light to Medium	Light
Iteration Duration	Long	Long	Short to Medium	Short to Medium	Medium to Long	Short	Short	Short
Development Speed	Slow	Slow	Moderate	Moderate	Moderate	Fast	Moderate	Fast
Testing	Post-development	Throughout Development	Continuous	Continuous	Continuous	Continuous	Continuous	Continuous
Best Suited For	Well-defined Requirements	Well-defined Requirements	Evolving Requirements	Complex Projects	Large and Complex Projects	Dynamic and Changing Requirements	Efficiency and Value-focused Projects	Continuous Deployment and Rapid Iteration

Architecture Simulation

Vending Machine Purchase:

https://gemini.google.com/share/2be5f7630142

The Reverse Engineering Lab

https://gemini.google.com/share/0bdba585a6b9